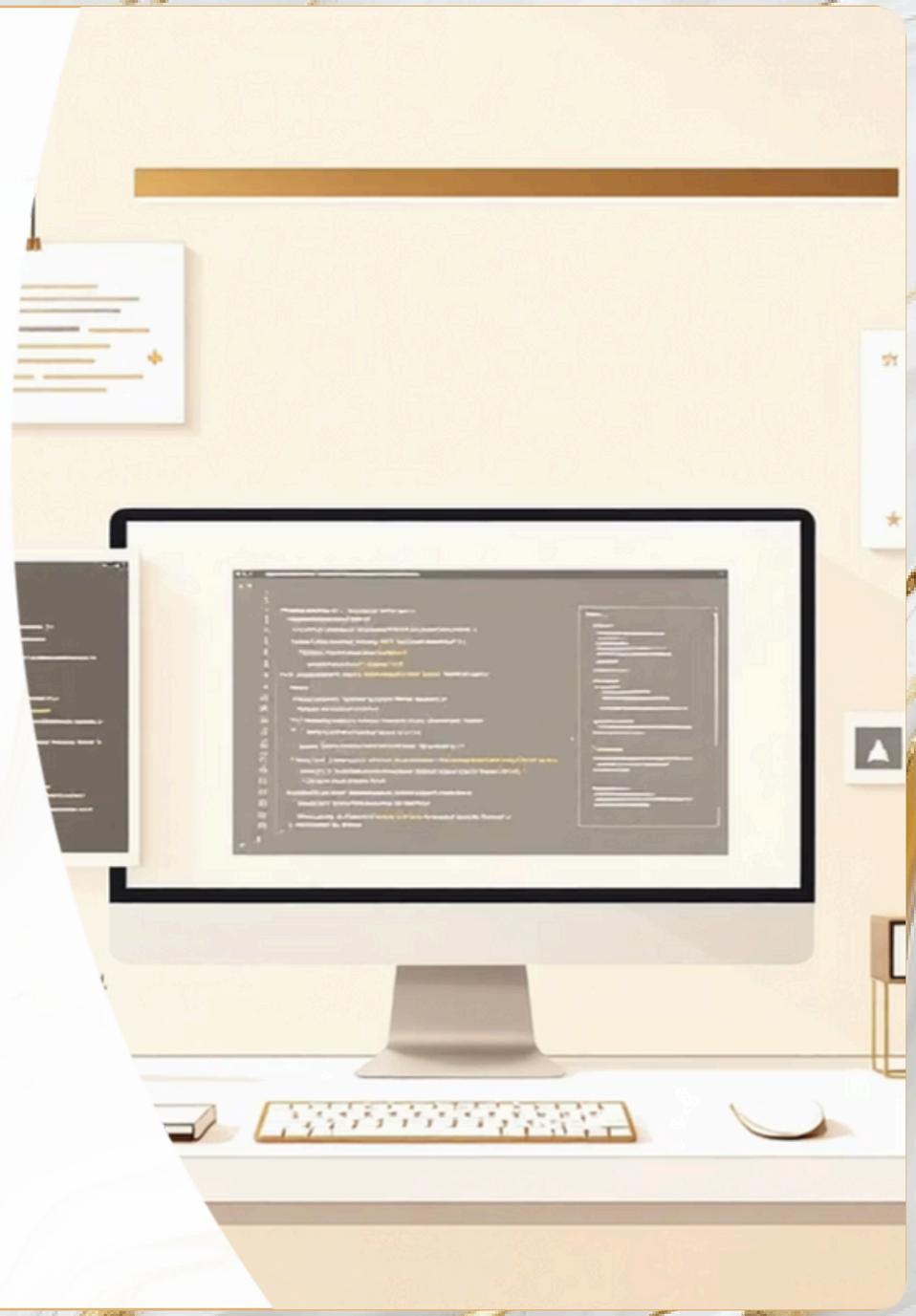
8. HAFTA: Yazılım Geliştirmede Kullanılan Araçlar

Yazılım geliştirme araçlarının evrimi, birbirinden bağımsız çalışan araçların oluşturduğu bir "araç zinciri" (toolchain) anlayışından, geliştirme yaşam döngüsünün her aşamasını kapsayan bütünleşik "DevOps platformları" anlayışına doğru belirgin bir paradigma kaymasını göstermektedir.



Geçmiş: İzole Araçlar

Başlangıçta, geliştiriciler kod editörü, derleyici ve dağıtım aracı gibi farklı görevler için izole programlar kullanıyordu.

- Bağımsız çalışan araçlar
- Manuel entegrasyon
- Sınırlı işbirliği

Bugün: Bütünleşik Platformlar

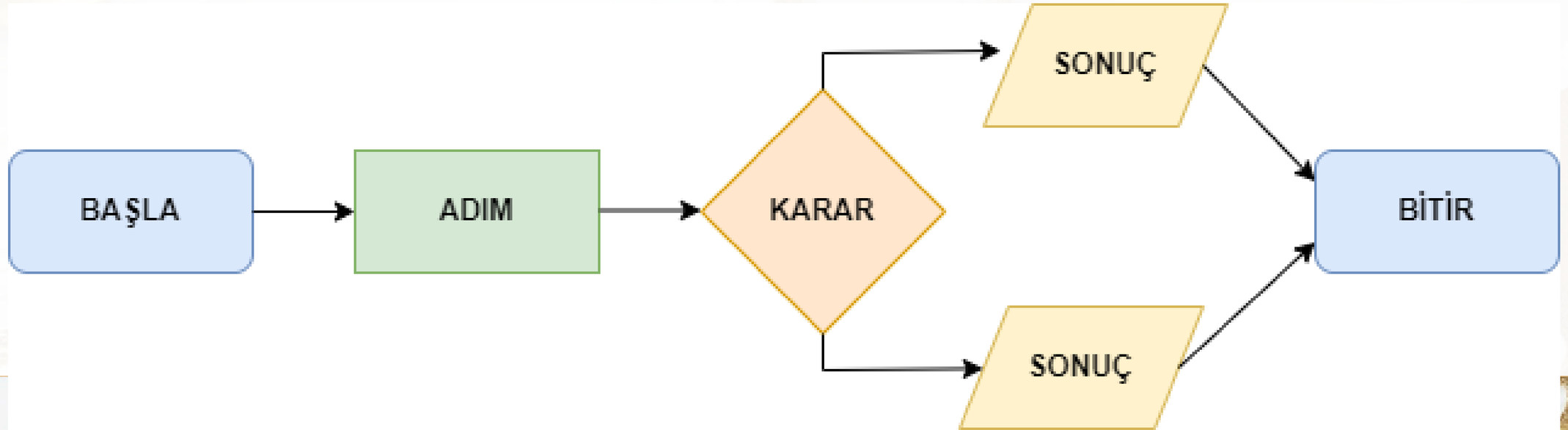
Geliştirme ve operasyon (DevOps) ekiplerini bir araya getiren işbirlikçi yaklaşımların yükselişi, bu araçların birbirleriyle sorunsuz bir şekilde çalışmasını gerektirdi.

- Tek arayüz
- Otomatik entegrasyon
- Sürekli işbirliği

Bu otomasyon ve entegrasyon ihtiyacı, GitLab gibi sadece versiyon kontrolü sunmakla kalmayıp aynı zamanda CI/CD ve proje yönetimi gibi özellikleri tek bir arayüzde birleştiren platformların doğuşuna zemin hazırlamıştır. Bu evrim, modern yazılım geliştirmenin temelinde yatan entegrasyon ve otomasyon felsefesini yansıtmaktadır.

Veri Akış Diyagramı (VAD)

SDLC'nin tasarım aşamasında, geliştirilecek sistemin mantıksal yapısını ve işleyişini anlamak için modelleme teknikleri kullanılır. Veri Akış Diyagramı (VAD veya Data Flow Diagram - DFD), bir sistem veya süreç boyunca verinin kaynaktan hedefe nasıl hareket ettiğini gösteren temel bir görsel modelleme aracıdır. 1970'lerde yapısal analiz metodolojilerinin bir parçası olarak ortaya çıkan VAD, sistemdeki veri akışını grafiksel olarak izah etmek için kullanılır.



Veri Akış Diyagramları'nın Dört Temel Bileşeni

Proses (Process)

Gelen veriyi işleyerek yeni bir çıktı verisi üreten bir fonksiyon veya aktiviteyi temsil eder.

Veri Akışı (Data Flow)

Sistemdeki bir noktadan diğerine hareket eden veriyi temsil eder ve bir ok ile gösterilir.

Veri Deposu (Data Store)

Sistemin veriyi sakladığı bir konumu (örneğin bir veritabanı tablosu veya bir dosya) temsil eder.

Dış Varlık (External Entity)

Sistemin etkileşimde bulunduğu, sistem dışındaki bir kişi, organizasyon veya başka bir sistemi temsil eder.

Veri Akış Diyagramları'nın Hiyerarşik Yapısı

Karmaşık sistemleri tek bir diyagramda ifade etmek zor olduğu için VAD'lar hiyerarşik bir yapıda katmanlara ayrılır.

Seviye 0 - Bağlam Diyagramı

En üst seviye olan Seviye 0 veya Bağlam Diyagramı (Context Diagram), tüm sistemi tek bir proses olarak gösterir ve dış varlıklarla olan temel veri alışverişini özetler.

Alt Seviyeler (Seviye 1, 2...)

Alt seviyeler (Seviye 1, Seviye 2 vb.), bu ana prosesi daha detaylı alt proseslere ayırarak sistemin iç işleyişini kademeli olarak ortaya koyar.

Önemli Not: Veri Akış Diyagramları, sistemin durağan yapısını, aşamalar arası veri akışını ve ana veri depolarını gösterirken; süreçlerin zamanlaması, karar yapıları veya döngüler gibi kontrol akışı bilgilerini göstermezler.

Mantıksal ve Fiziksel Veri Akış Diyagramları

Mantıksal VAD

Sistemin **"ne yaptığına"** odaklanarak iş süreçlerini ve veri akışlarını teknoloji bağımsız bir şekilde modeller.

- İş süreçleri odaklı
- Teknoloji bağımsız
- Kavramsal seviye

Fiziksel VAD

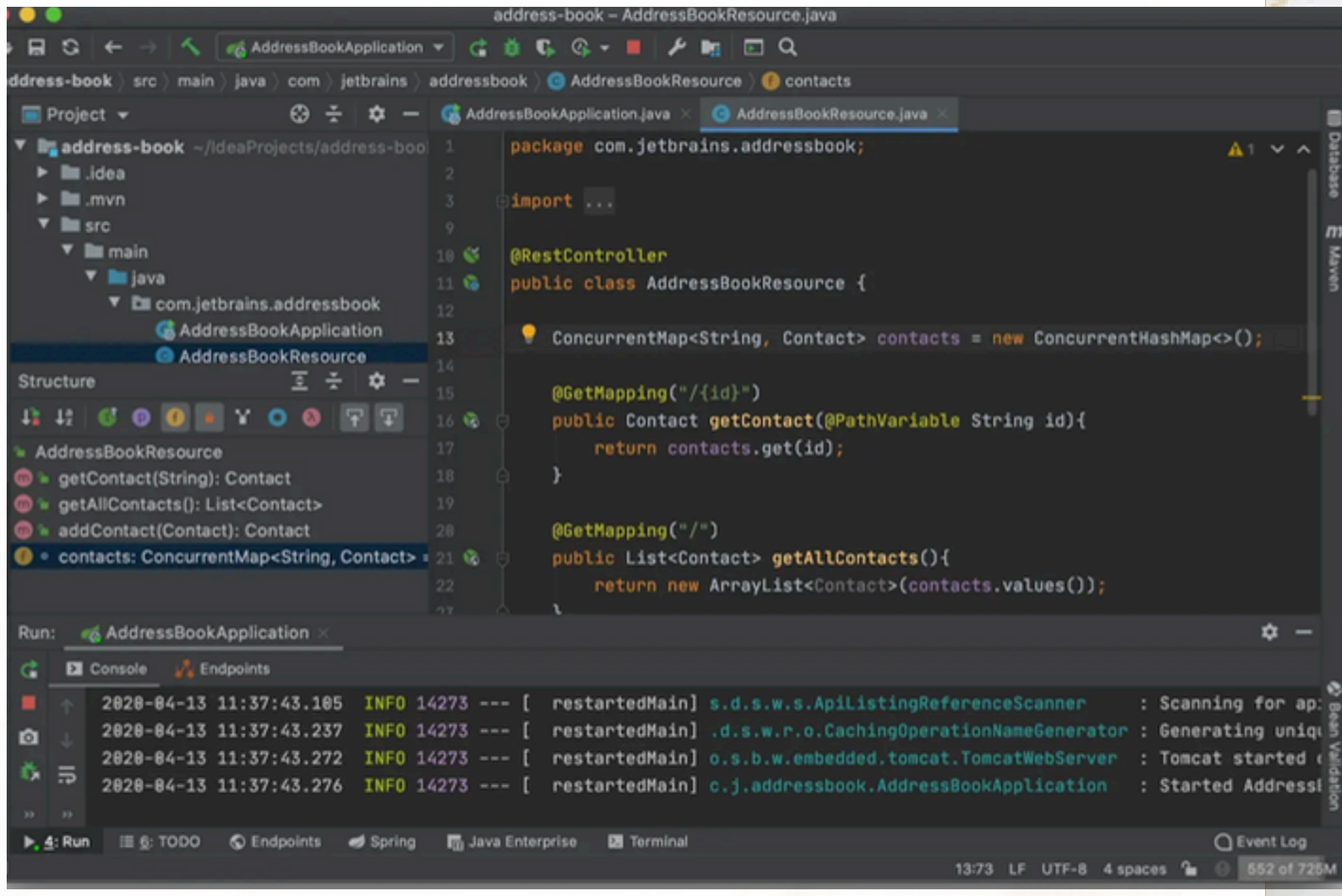
Bu süreçlerin **"nasıl uygulandığını"** göstererek belirli teknolojileri, dosyaları veya donanım bileşenlerini diyagrama dahil eder.

- Uygulama detayları
- Teknoloji spesifik
- Fiziksel seviye

Sistem analizi ve tasarımı için Microsoft Visio, Miro ve Canva gibi çeşitli araçlar, VAD oluşturmayı kolaylaştıran şablonlar, işbirliği özellikleri ve diğer platformlarla entegrasyon imkanları sunar.

Entegre(Tümleşik) Geliştirme Ortamları (IDE'ler)

Entegre Geliştirme Ortamı (Integrated Development Environment - IDE), yazılım geliştiricilerin kod yazma, düzenleme, hata ayıklama ve derleme gibi işlemleri tek bir arayüzde birleştiren kapsamlı bir yazılım uygulamasıdır. Geliştiricilerin üretkenliğini en üst düzeye çıkarmak için tasarlanan IDE'ler, geliştirme sürecinde ihtiyaç duyulan tüm temel araçları bir araya getirir.



IDE'nin Dört Temel Bileşeni ve Özellikleri

Kaynak Kodu Editörü

Gelişmiş bir metin düzenleyicidir. Kodun okunabilirliğini artırmak için anahtar kelimeleri, değişkenleri ve diğer dil öğelerini farklı renklerde gösteren sözdizimi vurgulama (syntax highlighting) özelliğine sahiptir. Ayrıca, geliştiricinin yazmakta olduğu kodu analiz ederek olası tamamlamaları öneren akıllı kod tamamlama (IntelliSense) gibi özelliklerle kodlama sürecini önemli ölçüde hızlandırır.

Derleyici ve Yorumlayıcı

Yazılan kaynak kodunu, bilgisayarın veya işletim sisteminin anlayabileceği makine koduna veya ara bir dile çeviren araçları içerir. Bu sayede geliştiriciler, yazdıkları kodu doğrudan IDE içinden çalıştırabilirler.

IDE'nin Dört Temel Bileşeni ve Özellikleri

Hata Ayıklayıcı (Debugger)

Yazılımdaki hataları bulmak ve düzeltmek için en kritik araçlardan biridir. Hata ayıklayıcı, programın belirli bir noktasında (breakpoint) durdurulmasına, kodun satır satır çalıştırılmasına, değişkenlerin değerlerinin anlık olarak incelenmesine ve hataların kaynağının tespit edilmesine olanak tanır.

Entegrasyonlar

Modern IDE'ler, Git gibi versiyon kontrol sistemleriyle, veritabanı yönetim araçlarıyla ve diğer geliştirme araçlarıyla sıkı bir entegrasyon sunar. Bu, geliştiricilerin farklı uygulamalar arasında geçiş yapma ihtiyacını ortadan kaldırır.

Popüler IDE'ler

Piyasada, farklı programlama dilleri ve platformlar için özelleşmiş çok sayıda popüler IDE bulunmaktadır:



Microsoft Visual Studio ve Visual Studio Code

Visual Studio, özellikle.NET ve C# ile Windows tabanlı uygulama geliştirmede güçlü bir standarttır. Visual Studio Code ise hafif, esnek yapısı ve geniş eklenti desteği ile neredeyse tüm diller için popüler hale gelmiş açık kaynaklı bir editör/IDE'dir.



JetBrains Ailesi

JetBrains tarafından geliştirilen IntelliJ IDEA, PyCharm, PhpStorm gibi IDE'ler, sırasıyla Java, Python ve PHP gibi dillere özel, son derece güçlü ve akıllı özellikleriyle tanınır.



Eclipse

Özellikle Java geliştirmede yaygın olarak kullanılan, açık kaynaklı ve eklentilerle yüksek düzeyde özelleştirilebilen bir IDE'dir.

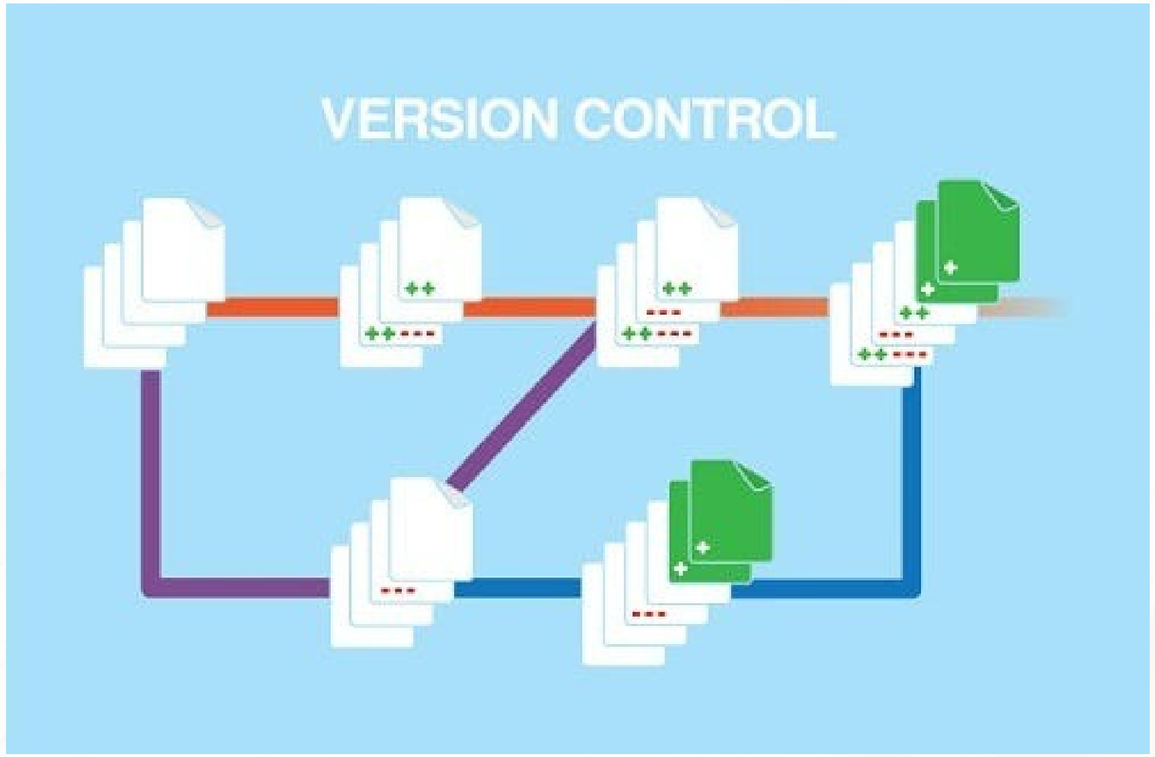


Xcode ve Android Studio

Sırasıyla Apple (iOS, macOS) ve Android ekosistemleri için uygulama geliştirmek üzere sunulan resmi ve kapsamlı geliştirme ortamlarıdır.

Versiyon Kontrol Sistemleri (VCS)

Versiyon Kontrol Sistemi (Version Control System - VCS), bir projedeki dosya ve klasörlerin zaman içindeki değişiklik geçmişini kaydeden ve yöneten bir sistemdir. Bu sistemler, yazılım geliştirme sürecinin temel bir parçasıdır ve yapılan her değişikliğin izlenmesini, karşılaştırılmasını ve gerektiğinde geri alınmasını sağlar.



VCS Kullanmanın Temel Faydaları

İzlenebilirlik ve Geçmiş Yönetimi

VCS, bir dosyada hangi değişikliğin, kim tarafından, ne zaman ve neden yapıldığının kaydını tutar. Bu, projenin evrimini anlamayı ve belirli bir değişikliğin kökenini bulmayı kolaylaştırır.

Ekip Çalışması ve İşbirliği

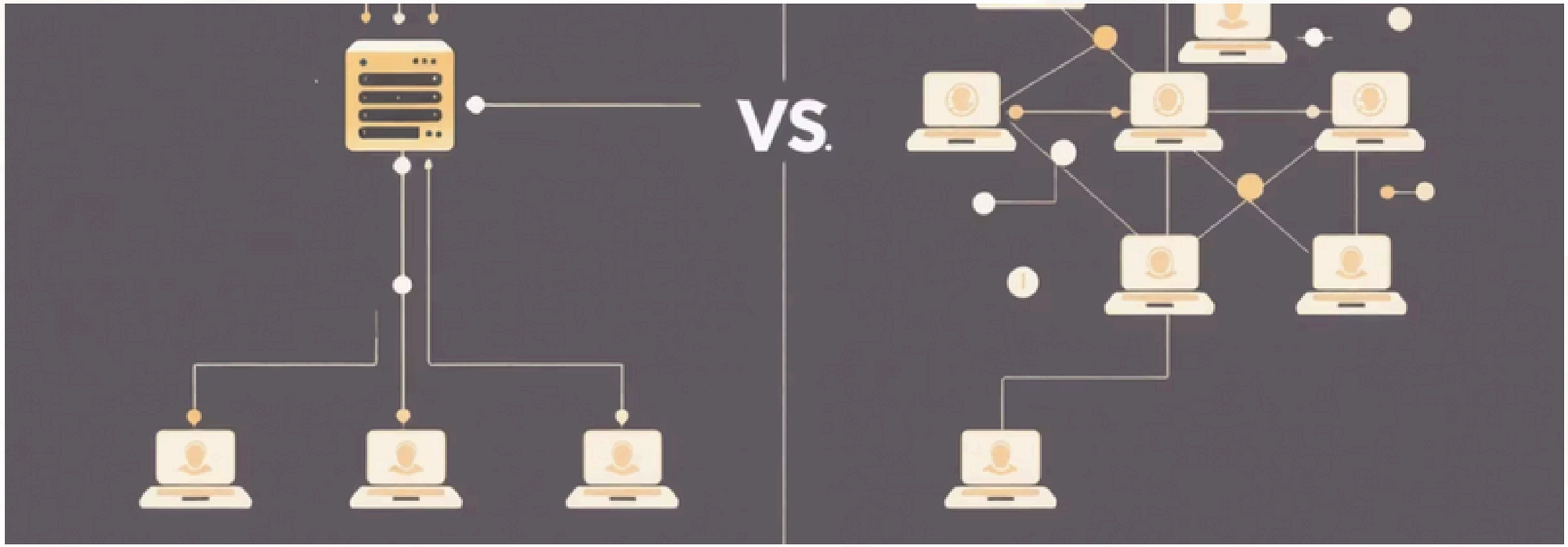
Birden fazla geliştiricinin aynı proje üzerinde eş zamanlı olarak çalışmasına olanak tanır. VCS, farklı geliştiricilerin yaptığı değişiklikleri birleştirmek (merge) ve olası çakışmaları (conflict) yönetmek için mekanizmalar sunar, böylece ekip üyeleri birbirlerinin işini ezmeden uyum içinde çalışabilir.

Hata Yönetimi ve Geri Alma (Rollback)

Bir değişiklik beklenmedik bir hataya yol açtığında, VCS projenin tamamını veya belirli dosyaları hatanın olmadığı önceki kararlı bir sürüme kolayca geri döndürme imkanı sunar. Bu, geliştirme sürecine büyük bir güvenlik ağı sağlar.

Versiyon Kontrol Sistemlerinin Tarihi

Versiyon kontrol sistemlerinin tarihi, 1970'lerde ve 1980'lerde ortaya çıkan Merkezi Versiyon Kontrol Sistemleri (Centralized Version Control Systems - CVCS) ile başlar. CVS ve Subversion (SVN) gibi sistemler, tüm sürüm geçmişini tek bir merkezi sunucuda saklardı. Daha sonra, Git gibi Dağıtık Versiyon Kontrol Sistemleri'nin (Distributed Version Control Systems - DVCS) ortaya çıkmasıyla bu alanda bir devrim yaşanmıştır.



Merkezi ve Dağıtık Sistemlerin Karşılaştırması: SVN vs. Git

Merkezi Mimari (SVN)

Bu modelde, projenin tüm versiyon geçmişini barındıran tek bir **merkezi sunucu** bulunur. Geliştiriciler, çalışmak istedikleri dosyaların bir kopyasını sunucudan kendi yerel makinelerine çekerler ("checkout"). Değişikliklerini tamamladıktan sonra, bu değişiklikleri doğrudan merkezi sunucuya gönderirler ("commit"). Bu mimari, bir istemci-sunucu modeline dayanır.

Dağıtık Mimari (Git)

Bu modelde, merkezi bir sunucuya ek olarak, her geliştiricinin kendi bilgisayarında projenin **tam bir kopyası (repository)** bulunur. Geliştiriciler, değişikliklerini önce bu yerel depoya "commit" ederler. Bu işlemler (commit, branch oluşturma vb.) tamamen yerel ve hızlıdır. Değişikliklerini ekiple paylaşmak istediklerinde ise yerel depolarını merkezi bir depoyla senkronize etmek için "push" (gönderme) ve "pull" (çekme) komutlarını kullanırlar.

Mimari Farklılıkların Pratik Sonuçları

Çevrimdışı Çalışma

Git'in en büyük avantajlarından biri, yerel bir depoya sahip olması sayesinde tam çevrimdışı çalışma yeteneğidir. Geliştiriciler, ağ bağlantısı olmadan projenin tüm geçmişini inceleyebilir, yeni commit'ler yapabilir ve dallar oluşturabilir. SVN'de ise commit gibi çoğu temel işlem için merkezi sunucuya aktif bir bağlantı gereklidir.

Performans

Git'te çoğu operasyon yerel disk üzerinde gerçekleştiği için ağ gecikmesi olmadan son derece hızlıdır. SVN, her işlem için merkezi sunucuyla iletişim kurduğundan daha yavaş olabilir. Ancak SVN, özellikle sık değişen büyük ikili (binary) dosyaların (örneğin tasarım dosyaları, 3D modeller) yönetiminde, sadece son sürümü indirdiği için Git'ten daha verimli olabilir.

Dallanma ve Birleştirme

Git'in dallanma modeli, bir dalın sadece belirli bir commit'e işaret eden hafif bir referans olması nedeniyle son derece hızlı ve verimlidir. Bu, geliştiricilerin yeni özellikleri denemek veya hataları düzeltmek için sık sık ve korkusuzca dallar oluşturmasını teşvik eder. SVN'de ise bir dal oluşturmak, sunucu üzerinde bir dizinin kopyalanması anlamına gelir ki bu daha hantal bir işlemdir. Ayrıca, SVN'de dalları birleştirmek genellikle Git'e göre daha karmaşık ve çakışmaya yatkındır.

Veri Bütünlüğü ve Güvenlik

Git: Kriptografik Güvenlik

Git, her commit'i, dosyayı ve dizini kriptografik bir özet fonksiyonu olan SHA-1 ile imzalayarak projenin geçmişinin bütünlüğünü garanti altına alır. Bu, geçmişin izinsiz değiştirilmesini neredeyse imkansız hale getirir.

SVN: Granüler Erişim Kontrolü

SVN ise daha granüler erişim kontrolü sunar; belirli dosya veya dizinlere kullanıcı bazında okuma/yazma izinleri tanımlanabilir, bu da merkezi kontrol gerektiren kurumsal ortamlarda bir avantaj olabilir.



Git ve GitHub Arasında Temel Fark

Git

Git'in kendisi, temel olarak komut satırından kullanılan güçlü bir araçtır. Ancak modern yazılım geliştirmede işbirliği, kod incelemesi ve proje yönetimi gibi ek işlevler de kritik öneme sahiptir.

Git, versiyon kontrolünü gerçekleştiren dağıtık bir sistemdir.

GitHub

GitHub, bu ihtiyacı karşılamak üzere ortaya çıkmış, Git üzerine inşa edilmiş web tabanlı bir hizmet ve kod barındırma platformudur. Git'in teknik gücünü alıp üzerine sosyal ve işbirlikçi özellikler ekleyerek geliştirme süreçlerini dönüştürmüştür.

GitHub ise bu Git depolarını bulutta barındıran, yöneten ve ekiplerin bu depolar üzerinde işbirliği yapmasını sağlayan bir platformdur.

GitHub'ın Sunduğu Temel Özellikler

Kod Barındırma (Repository Hosting)

Geliştiricilerin Git depolarını genel veya özel olarak bulutta güvenli bir şekilde saklamasına olanak tanır.

GitHub Actions

Doğrudan depo içinde CI/CD gibi otomasyon iş akışları oluşturmayı ve çalıştırmayı sağlayan güçlü bir özelliktir.

Pull Request (PR)

GitHub'ın en önemli işbirliği mekanizmasıdır. Bir geliştirici, yaptığı değişiklikleri projenin ana dalına birleştirmeden önce bir "Pull Request" açar. Bu, diğer ekip üyelerinin önerilen değişiklikleri satır satır incelemesine (code review), yorum yapmasına, tartışmasına ve onaylamasına olanak tanır. Bu süreç, kod kalitesini artırır ve hataların erken tespit edilmesini sağlar.

Issue Tracking

Projeyle ilgili hataların (bugs), özellik taleplerinin ve diğer görevlerin oluşturulması, atanması ve takip edilmesi için kullanılır. Bu, yapılandırılmış bir proje yönetimi sağlar.

Wiki ve Pages

Her depo için proje dokümantasyonu oluşturmak üzere bir Wiki bölümü ve statik web siteleri yayınlamak için Pages özelliği sunar.

Sürekli Entegrasyon ve Sürekli Dağıtım

CI/CD Prensibi

Sürekli Entegrasyon ve Sürekli Dağıtım (CI/CD), modern yazılım geliştirmenin temelini oluşturan, yazılımı hızlı, güvenilir ve tekrarlanabilir bir şekilde teslim etmeyi amaçlayan bir dizi pratiktir. Bu metodoloji, geliştirme ve operasyon ekipleri arasındaki işbirliğini artırarak ve süreçleri otomatikleştirerek yazılım teslimatını hızlandırır.



Sürekli Entegrasyon (Continuous Integration - CI)

Geliştiricilerin kod değişikliklerini sık sık (genellikle günde birden çok kez) paylaşılan bir kod deposuna entegre etme pratiğidir. Her entegrasyon, otomatik bir derleme (build) ve test sürecini tetikler. CI'ın temel amacı, birden fazla geliştiricinin yaptığı değişikliklerin birbiriyle uyumlu olup olmadığını erken bir aşamada tespit etmek ve entegrasyon sorunlarını ("entegrasyon cehennemi") önlemektir.

Sürekli Teslimat (Continuous Delivery - CD)

CI sürecinin doğal bir uzantısıdır. Bu yaklaşımda, kod değişiklikleri otomatik olarak derlenip test edildikten sonra, üretim ortamına dağıtıma hazır bir "aday sürüm" (release candidate) olarak paketlenir. Bu sürüm, ek testlerden (örneğin, kullanıcı kabul testleri) geçirildikten sonra tek bir manuel onayla (bir butona basarak) canlıya alınabilir. Amaç, yazılımın her zaman dağıtıma hazır olmasını sağlamaktır.

Sürekli Dağıtım (Continuous Deployment - CD)

Sürekli Teslimat'ın en ileri seviyesidir. Bu modelde, CI/CD hattının (pipeline) tüm aşamalarından başarıyla geçen her kod değişikliği, hiçbir insan müdahalesi olmadan otomatik olarak üretim ortamına dağıtılır. Bu, yeni özelliklerin ve hata düzeltmelerinin kullanıcılara çok daha hızlı bir şekilde ulaştırılmasını sağlar.

CI/CD'nin Temel Faydaları



Hızlı Geri Bildirim ve Azalan Risk

Otomatik testler sayesinde hatalar, geliştirme döngüsünün çok erken aşamalarında tespit edilir. Bu, hataların maliyetini düşürür ve daha küçük, artımlı değişiklikler yapıldığı için her sürümün riskini azaltır.



Otomasyon ve Verimlilik

Derleme, test etme ve dağıtma gibi manuel, tekrarlayan ve hataya açık süreçleri otomatikleştirir. Bu, geliştiricilerin zamanını daha değerli görevlere odaklamasını sağlar ve insan hatası riskini ortadan kaldırır.



Daha Güvenilir ve Sık Sürümler

Otomasyon ve sürekli testler, her sürümün kalitesini artırır. Bu güven, ekiplerin daha sık sürüm çıkmasını teşvik eder, bu da pazar taleplerine daha hızlı yanıt verme yeteneği kazandırır.

CI/CD Araçları

Jenkins

Açık kaynaklı bir otomasyon sunucusudur ve CI/CD alanındaki en eski ve en yaygın kullanılan araçlardan biridir. En büyük gücü, binlerce eklentiden (plugin) oluşan devasa ekosistemidir. Bu eklentiler sayesinde Jenkins, hemen hemen her türlü derleme, test ve dağıtım senaryosuna uyarlanabilir. Genellikle kendi sunucularınızda barındırılır (self-hosted), bu da tam kontrol ve esneklik sağlar.

GitLab CI

Kod deposu, proje yönetimi ve CI/CD'yi tek bir entegre arayüzde sunar. Yapılandırma, proje kök dizininde bulunan .gitlab-ci.yml adlı bir YAML dosyası aracılığıyla yapılır. Bu "her şey kodda" yaklaşımı, CI/CD süreçlerinin de kod gibi versiyonlanmasını ve yönetilmesini sağlar. GitLab, kendi "runner"larınızı (işleri yürüten ajanlar) kurmanıza olanak tanıdığı gibi, bulut tabanlı paylaşımlı runner'lar da sunar.

Travis CI

Özellikle açık kaynaklı GitHub projeleriyle popülerleşmiş, tamamen bulut tabanlı bir hizmettir (SaaS - Software as a Service). Kurulum ve bakım gerektirmez. Projenize .travis.yml adlı bir YAML dosyası ekleyip GitHub hesabınıza bağlamanız, CI sürecini başlatmak için yeterlidir. Bu basitliği ve GitHub ile sorunsuz entegrasyonu, onu birçok proje için cazip bir seçenek haline getirir.

Docker: Konteynerleştirme Platformu

Konteynerleştirme, bir uygulamayı ve onun tüm bağımlılıklarını (kütüphaneler, ayar dosyaları vb.) "konteyner" adı verilen tek bir standartlaştırılmış, izole ve taşınabilir birim içinde paketleme teknolojisidir. Docker, bu teknolojiyi yaygınlaştıran ve endüstri standardı haline getiren açık kaynaklı bir platformdur.

"Benim makinemde çalışıyordu" sorununu çözmek Docker'ın en temel vaadidir.

Bir uygulama, geliştiricinin bilgisayarındaki farklı kütüphane sürümleri veya işletim sistemi ayarları nedeniyle test veya üretim sunucusunda farklı davranabilir. Docker, uygulamayı kendi ortamıyla birlikte paketleyerek, bu konteynerin Docker'ın çalıştığı her yerde (geliştirici bilgisayarı, test sunucusu, bulut) tam olarak aynı şekilde çalışmasını garanti eder.

Konteynerler vs Sanal Makineler

Sanal Makineler (VM)

Bir VM, kendi tam işletim sistemini (Guest OS) içeren, donanım seviyesinde sanallaştırılmış bir yapıdır.

Konteynerler

Konteynerler ise, üzerinde çalıştıkları ana makinenin işletim sistemi çekirdeğini (Host OS Kernel) paylaşırlar. Bu sayede, her konteynerin kendi işletim sistemini çalıştırma yükü ortadan kalkar. Bu mimari fark, konteynerleri VM'lere göre çok daha hafif, hızlı (saniyeler içinde başlarlar) ve kaynak verimli hale getirir.

Docker’ın Temel Özellikleri

Dockerfile

Bir Docker imajının nasıl oluşturulacağını adım adım tanımlayan, metin tabanlı bir talimat dosyasıdır. Hangi temel imajın kullanılacağı, hangi dosyaların kopyalanacağı, hangi komutların çalıştırılacağı gibi bilgileri içerir.

Image (İmaj)

Bir uygulamanın çalışması için gereken her şeyi içeren, salt okunur bir şablondur. Bir Dockerfile'dan docker build komutu ile oluşturulur.

Container (Konteyner)

Bir imajın çalıştırılabilir bir örneğidir (instance). Bir imajdan docker run komutu ile oluşturulur ve uygulamanın çalıştığı izole ortamdır.

Docker Hub/Registry

Docker imajlarının saklandığı, paylaşıldığı ve yönetildiği bir depo hizmetidir. Docker Hub, en bilinen genel registry'dir.

Büyük Ölçekte Yönetim: Kubernetes ile Konteyner Orkestrasyonu

Kubernetes, konteynerleştirilmiş uygulamaların dağıtımını, ölçeklendirilmesini ve yönetimini otomatikleştiren, açık kaynaklı ve endüstri standardı haline gelmiş bir orkestrasyon sistemidir. Başlangıçta Google tarafından geliştirilen Kubernetes, bir “pilot" gibi, konteyner filonuzu yönetir.

Docker ve Kubernetes birbirinin rakibi değil, tamamlayıcısıdır. Docker, konteynerleri oluşturur ve paketler; Kubernetes ise bu Docker konteynerlerini büyük ölçekte yönetir ve organize eder. Kubernetes, altta yatan sunucu kümesini (cluster) yönetir ve Docker gibi bir konteyner motorunu kullanarak bu sunucular üzerinde konteynerleri çalıştırır.

Kubernetes'in Temel Orkestrasyon Yetenekleri

Otomatik Dağıtım ve Ölçeklendirme

Uygulama talebine veya CPU kullanımına göre konteyner sayısını otomatik olarak artırıp (scale-out) azaltabilir (scale-in).

Self-Healing (Kendini İyileştirme)

Çalışan bir konteyner çöktüğünde veya yanıt vermez hale geldiğinde, Kubernetes bunu otomatik olarak algılar ve yerine sağlıklı yeni bir tane başlatarak uygulamanın kesintisiz çalışmasını sağlar.

Hizmet Keşfi ve Yük Dengeleme

Bir grup özdeş konteynere (örneğin, bir web sunucusunun kopyaları) tek bir sabit ağ adresi (DNS adı) üzerinden erişim sağlar ve gelen trafiği bu konteynerler arasında akıllıca dağıtır.

Otomatik Güncellemeler ve Geri Almalar

Uygulamanın yeni bir sürümünü, sıfır kesintiyle (rolling update) aşamalı olarak dağıtabilir. Eğer yeni sürümde bir sorun tespit edilirse, aynı kolaylıkla önceki kararlı sürüme geri dönebilir.

UYGULAMA

- "Bir online eğitim platformuna 'kişiselleştirilmiş ders önerisi' özelliği eklenmesi senaryosunu düşünün. Bu özelliği geliştirmek ve yayına almak için bir DevOps iş akışı tasarlayın. Aşağıdaki adımları takip ederek akışınızı tanımlayın ve açıklayın.
- Tasarım ve Geliştirme: Hangi araçlar/ortamlar kullanılır ve neden?
- Versiyon Kontrolü: Kod değişikliklerini nasıl yönetirsiniz?
- İşbirliği ve Kod İncelemesi: Ekip nasıl dahil olur?
- Sürekli Entegrasyon (CI): Otomatik testler nasıl tetiklenir?
- Konteynerleştirme: Uygulama nasıl paketlenir?
- Sürekli Dağıtım (CD): Değişiklikler nasıl production'a taşınır?
- Orkestrasyon ve Dağıtım: Sıfır kesinti için ne yapılır?